A Hybrid Data-Centric and Model-Centric Approach Towards Robust Single-Lead ECG Obstructive Sleep Apnea Diagnosis

Quoc H. Nguyen[a], Chau Le[b], Chuan Pham[c], Minh N. H. Nguyen[d], Tien-Dung Nguyen[e], Arveity Setty[f], Trung Quoc Le[a,*]

[a] *Department of Industrial and Management Systems Engineering, University of South Florida, Tampa, 33559, USA*
[b] *Department of Civil, Construction and Environmental Engineering, North Dakota State University, Fargo, 58102, USA*
[c] *Synchromedia - École de Technologie Supérieure, Université du Québec, Québec, H3C1K3, Canada*
[d] *Computer Engineering and Electronics, Vietnam-Korea University of Information and Communication Technology, Da Nang, 550000, Vietnam*
[e] *Computer Science and Engineering, Kyung Hee University, Yogin, 17104, Korea*
[f] *School of Medicine and Health Sciences, University of North Dakota, Grand Forks, 58203, USA*

Abstract

Although existing model-based approaches to detect OSA events have been widely studied, such modeling efforts have faced considerable challenges, such as inconsistencies among expert interpretations and the high cost of model development due to insufficient input data. In addition, most OSA diagnostic models were built using multiple bio-signals collected by a polysomnography (PSG) system, which is costly and incompatible with building models that can be run on IoT devices for at-home monitoring and treatment solutions. This study presents a hybrid data-centric and model-centric framework for maximizing operational capability and improving reliability in recognizing OSA events using single-lead ECG. Specifically, data-centric strategies were

*Corresponding author

Email addresses: nguyenq29@usf.edu (Quoc H. Nguyen), chau.le@ndsu.edu (Chau Le), chuan.pham@etsmtl.ca (Chuan Pham), nhnminh@vku.udn.vn (Minh N. H. Nguyen), ntiendung@khu.ac.kr (Tien-Dung Nguyen), arveity.setty@sanfordhealth.org (Arveity Setty), tqle@usf.edu (Trung Quoc Le)

developed to extract and refine critical features for OSA diagnosis. Based on the extracted quality features from the data-centric component, a LSTM-based model was developed for detecting OSA. The hyper-parameters tuning, model validation and verification were all examined. In addition, the model generalization was evaluated on different datasets from the Physionet Apnea-ECG database and a pre-clinical IRB approved study. The results showed that the hybrid framework effectively recognized OSA events and outperformed baseline models. In practice, the work provides an efficient solution to building lightweight and effective ML models at the edges.

Keywords:
Data-Centric, Model-Centric, Sleep Apnea Diagnosis, Stacked LSTM, Single-lead ECG

---

1. Introduction

Sleep Apnea (SA) is a severe chronic sleep disorder characterized by brief pauses in breathing throughout the night. Obstructive Sleep Apnea (OSA), which represents the great preponderance of the cases, when left untreated, can lead to high blood pressure, chronic heart failure, atrial fibrillation, stroke, and other cardiovascular problems in both children and adults [1]. Unfortunately, according to the American Sleep Association, it is estimated that 25 million Americans [2] and over 1 billion people worldwide suffered from OSA in 2021, with 80% of the cases of moderate and severe OSA undiagnosed [3]. In addition, OSA patients have a 2.93 times higher risk of being hospitalized when affected by COVID-19 than non-OSA individuals, resulting in rising mortality and treatment costs [4, 5]. This demonstrates the importance of a precise OSA diagnosis in order to provide timely treatment options.

Polysomnography (PSG) is currently recognized as the most popular and effective tool for diagnosing OSA. However, the implementation of PSG requires OSA subjects to sleep for one or two nights in a dedicated sleep laboratory with special equipment, to have many wires attached to the subject body, as well as continuous assistance from medical technicians. The inconvenience, cost, and discomfort of PSG create a barrier to its widespread acceptance by the public. To address this issue, researchers have proposed a number of OSA detection methods based on various types of single-channel signals, including electrocardiogram (ECG), electroencephalogram (EEG),

snoring, and respiratory signals [6, 7, 8, 9]. These single-channel signals are easily collected by IoT wearable devices [10]. In addition, previous works have shown that the RR-interval and R-peak amplitude features derived from the single-lead ECG signal contain essential information on OSA occurrence [11, 12]. Therefore, AI-based models for OSA diagnosis at the edge devices trained using extracted RR-intervals and R-peak amplitudes are an ideal solution that provides comfort and portability in data collection setup, as well as lightweight models and cost efficiency in model construction. For example, OSA detection models can be directly embedded in IoT devices where sing-lead ECG data is collected and processed to provide real-time diagnosis rather than costly analysis of data collected from the PSG system.

Existing model-based approaches for identifying OSA events and model-centric AI in general have been intensively investigated. However, the model development requires massive datasets and expensive computational resources in order to achieve performance improvements. These requirements will become a problem when training data is limited and difficult to collect due to security and privacy constraints, especially in the healthcare industry. Unfortunately, the importance of data gathering and training preparation are often neglected in AI development. Because of this, it takes hundreds of hours to fine-tune a model that was trained on noisy and poorly labeled data. So, it is important to find a strategy that can address the aforementioned challenges. Fortunately, Andrew Ngo's "data-centric AI" approach, which is the discipline of systematically engineering the data used to develop an AI system, can help to build AI models with limited datasets and make model interpretation more transparent and efficient. The fundamental idea is to concentrate on enhancing the quality and consistency of the training data rather than just improving the code (i.e., algorithm/model) [13, 14]. By focusing on data quality, there is a greater possibility of eliminating data bias through careful analysis and preventing the drifting concept in model-centric AI, which refers to the change in input-output relationships over time [15, 16]. In addition, data-centric AI focuses on data quality rather than data quantity and does not require expensive computational resources to process a large amount of data. Therefore, "small data" solutions with data-centric AI can satisfy major requirements such as model efficiency, accuracy, and bias, leveraging AI applications at the edge. In fact, developing a hybrid-centric system is a great approach to elaborate robust AI development, because it can utilize the advantages of data-centric AI while employing well-developed methods in model-centric AI. For example, by using a hybrid-centric ap-

proach, the data-centric strategies can be used to improve the quality of OSA and non-OSA annotations (i.e., making a standard training dataset), while the model-centric strategies can be used to enhance the model selection based on the standard training dataset from the data-centric approach. Such combinations facilitate the characterization and learning of the underlying processes of the data acquired by wearable devices.

1.1. Related Works

There has been considerable research on the development of algorithms for the detection of sleep apnea from a variety of physiological signals [17, 18, 19, 20], among which ECG has been the most significant signal due to its pathophysiological relevance and readily accessible in-lab polysomnography or at-home sleep screening systems. Bsoul et al. [21] observed that ECG signals have a significant influence on the identification of OSA and employed a real-time sleep apnea monitoring algorithm using single-lead ECG signal. Nguyen and Kunal et al. [22, 23] presented a sleep apnea diagnosis system based on heart rate complexity calculated by the heart rate variability (HRV) information extracted from the ECG signals. In another work, Hasan et al. [24] implemented an approach where spectral features were extracted from single-lead ECG signals and then bootstrap aggregating was used to classify sleep apnea. In another study, Nishad et al. [25] used tunable-Q wavelet transformation to decompose the ECG signals and later extracted entropy features to classify sleep apnea using various classifiers. Martin et al. [26] proposed a feature extraction technique using HRV information captured from the ECG signals and later classified sleep apnea using Linear Discriminant Analysis (LDA), Quadratic Discriminant Analysis (QDA), and Logistic Regression. Varon et al. [27] proposed to extract two novel features from HRV information and use least-squares support vector machines (LS-SVM) to detect sleep apnea events in ECG signals. The aforementioned ML sleep apnea detection approaches show that apnea detection using single-lead ECG provides an opportunity to design and develop new technologies on wearable devices and sensors [28, 29, 30, 31]. However, most of the works usually focus on how to extract relevant features that require rich experience and specialized knowledge from experts. Plus, there is no hybrid between data-centric and model-centric implemented, even though some works have mentioned improving input data via pre-processing, which is not fully reached the potential of other data-centric methods.

In recent years, OSA detection based on deep learning (DL) has been applied and achieved good results. Wang et al. [32] described a residual network to analyze insights in RR Intervals to detect apnea segments. In another work, Wang et al. [33] applied a modified version of the LeNet-5 convolutional neural network (CNN) to identify apnea events. Feng et al. [34] proposed an unsupervised feature learning deep network model of a frequency-domain stacked SAE (FSSAE) and successfully introduced the unsupervised learning method into the comprehensive feature extraction of sleep apnea. Additionally, many other research studies used DL to automatically learn high-level semantic features to detect OSA from raw ECG signals. Such models can be used to tackle the problem of manual feature engineering, in which the feature extraction process relies mainly on domain expertise [35, 36, 37]. However, there are pitfalls to being excessively fixated on OSA detection models. Reproducing results for state-of-the-art deep neural network methods is seldom straightforward. In particular, non-determinism in typical benchmark contexts combined with intrinsic variance in the methods can make reported results difficult to interpret. As a result, obtaining a state-of-the-art model-centric approach incentivizes inane practices like fixing random seeds or using the same reporting methodologies, both of which are costly and time-consuming. One of the primary causes of this problem is that the dataset is not standard and consistent [38]. Therefore, a data-centric strategy for generating standard input data is needed. Then, reproducing and analyzing results will be more accessible, reliable, and transparent.

Handcrafted features created by human experts with prior expertise and domain knowledge have been utilized in a data-centric manner to build end-to-end deep learning (DL) models. Especially, feeding handcrafted features extracted from limited healthcare data into DL models can increase the model interpretability and performance [39, 11]. According to Guilleminault et al. [40], there is a relation between OSA events and heart rate variability. It has been indicated that the heart rate decelerates at the beginning of an OSA event and suddenly increases when normal breathing is resumed. To learn the characteristics of temporal features from sequences of inputs in the data, long short-term memory (LSTM) structure has been used to capture the internal memory and utilizes feedback connections of the underlying system [41, 42, 43]. In this study, we propose a customized stacked LSTM adoption to learn the feature dependencies, such as the transition rules employed by physicians, to identify future OSA events from previous ECG intervals.

1.2. Motivations and Key Contributions

Code and data both play an important role in AI development. There are no fixed rules for choosing between model-centric and data-centric AI, but the robustness of the dataset should not be disregarded while deploying these systems. In addition, AI-based models at the edge using single-lead ECG data have great potential to be used in developing at-home sleep monitoring systems with wearable technologies. They can be rendered on a large scale with decentralized Federated Learning (FL) applications [44], where portable and lightweight AI models are required at the edges to collaboratively train a global model without compromising data privacy. Inspired by the first "Data-Centric AI" competition conducted and organized by Andrew Ng [13], we propose a hybrid data-centric and model-centric approach that can fully characterize domain knowledge-based features throughout the data-centric process while utilizing the model optimization and selection mechanisms of the model-centric process.

The main contributions are as follows:

- To the best of our knowledge, we propose a universal OSA diagnosis system as the first hybrid data-centric and model-centric mechanism for detecting OSA based on single-lead ECG. A broad perspective on enhancing data quality based on domain expertise is provided. This brand-new and promising hybrid perspective improves the resilience of deep networks in the aspect of data and greatly enriches the research community.

- We analyse the standard LSTM to provide extensive instruction on how to build a detection learning network by leveraging stacked LSTM with handcrafted features. This serves the purpose of understanding and re-engineering the model so that it can be applied in different related problems.

- We provide an open-source framework for the development of hybrid-centric systems and a detailed analysis of the experimental outcomes to evaluate the system's efficiency with baseline methods. The source code is available at [45].

The rest of the paper is organized as follows. Section II describes the dataset and introduces strategies of the hybrid approach. Section III presents the experimental results. In Section IV, discussions are presented. Finally, we

draw conclusions regarding the potential usefulness of this system in Section V.

2. Material and Methodology

We first describe the dataset used for OSA diagnosis and then analyze essential aspects of the hybrid data-centric and model-centric system.

2.1. Dataset Description

The Physionet Apnea ECG dataset (PAED) was used to compile the data for this work [46]. It consists of 70 nighttime recordings of single continuous ECG signals with a sampling rate of 100Hz, 160-bit resolution, and a duration of 8 hours per night. Experts segmented the recordings on a minute-by-minute basis and labeled each segment as normal or apnea. In this investigation, recordings with $AHI \geq 5$ were classified as OSA positive; otherwise, they were classified as OSA negative. In addition, we utilized an-

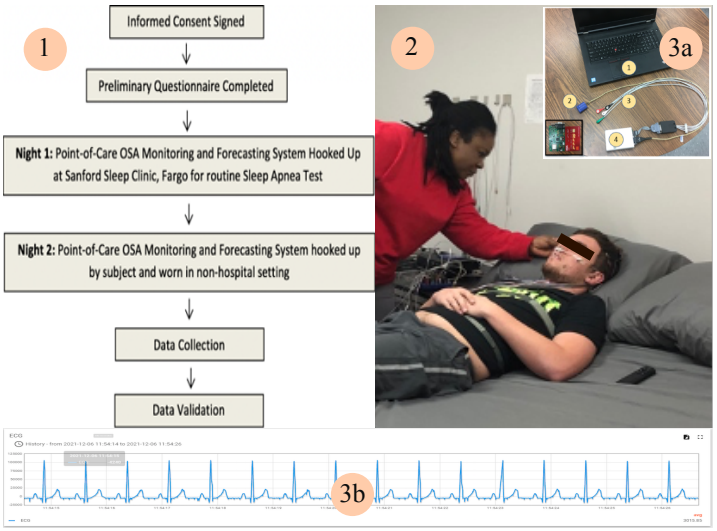


Figure 1: An IRB-approved design and implementation of a data collection session on cancer patients. 1) Data collection protocol; 2) Measurement setup; 3a) Single-lead ECG device, and 3b) Real-time ECG collected signal.

other Sleep Apnea Dataset (SAD) collected from cancer patients referred to Sanford Sleep Center for the validation of the proposed model. The data collection protocol, measurement setup, and system configuration are described

in Fig. 1. The dataset consists of 5 patient records collected over the period of a 7-hour night at a sampling rate of 256Hz. An expert at Sanford annotated the apnea events at 30-second intervals. To match with the PAED, SAD was pre-processed with a labeling procedure to convert 30-second to one-minute intervals. Table. 1 shows the key statistics of two datasets.

Table 1: Baseline characteristics of the patients from the Apnea-ECG database (PAED) and our dataset collected and pre-processed from Sanford Health Hospital (SAD).

Measures (units)	**PAED Mean ± SD**	**SAD Mean ± SD**
Records	70	5
Duration (min)	490.1 ± 31.5	439.4 ± 93.1
Apnea/hypoapnea (min)	186.6 ± 170.9	88.4 ± 54.2
AHI (events/hr)	28.0 ± 27.3	11.0 ± 13.2
Age (yr)	45.1 ± 10.7	57 ± 7.4
Males	57	1

AHI: apnea-hypoapnea index.

2.2. Hybrid-centric System Overview

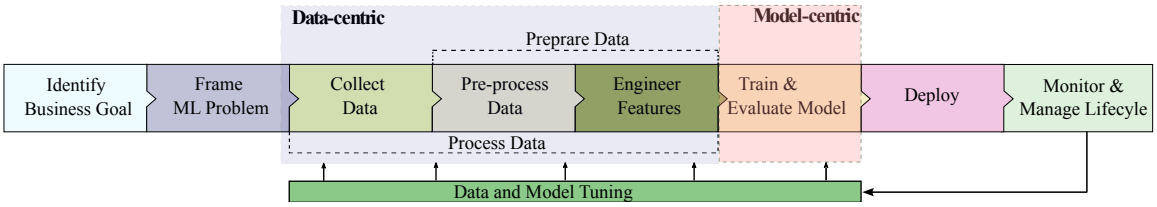


Figure 2: The hybrid-centric approach life cycle.

Fig. 2 illustrates the proposed architected of ML life cycle, where data-centric and model-centric parts are being monitored and tuned to ensure the quality of the whole system. We designed it in a generic manner to be employed in any ML development scheme. In the context of OSA diagnosis, after identifying business objectives and formulating the diagnosis problem based on the single-lead ECG, the raw input data passed through the data processing procedure and was transformed into a usable format. As shown in Fig. 2, the data-centric strategy enhances input data. It is the process of constructing an iteratively evolving machine learning system in which data is continuously refined over iterations. Data processing is the main task in data-centric solutions, which includes data collection and data preparation

to create, transform, extract, and select insights from raw input data. Fig. 3 depicts data processing components. In addition to the data-centric part, the model-centric component includes model training and evaluation, where the models are trained, tuned, and evaluated before being deployed into production. Both data-centric and model-centric components are monitored by the monitor and manage life cycle module to achieve the desired level of system performance. The model output is sent backward through the data and model tuning module to the data processing and model training stages to constantly enhance and correct issues in the input data and prior models.

2.3. Data-centric Approach

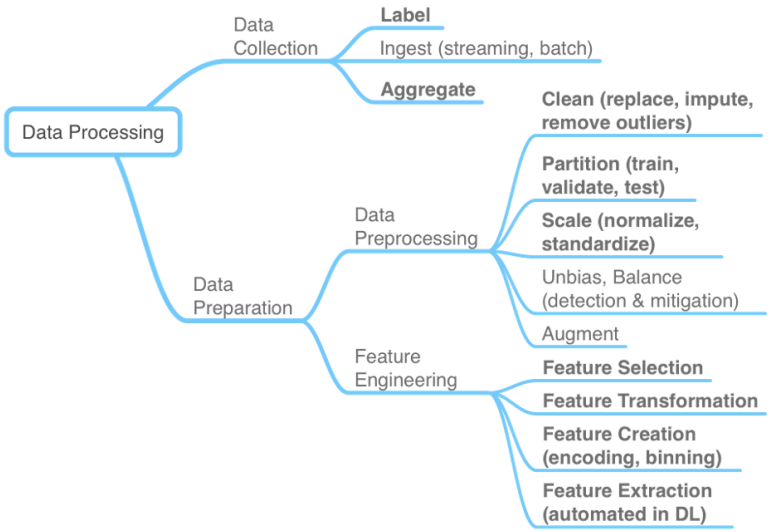


Figure 3: Data processing components. The implemented steps to process the data for OSA detection are bold.

We did not perform all of the items in the proposed data processing for the OSA case study; instead, the highlighted components in Fig. 3 were chosen to implement. They are explained in the following subsections. In general, the more efficient the data processing procedures, the better the input data for the ML models.

2.3.1. OSA Labeling Strategy

Labeling, ingesting, and aggregating data from multiple data sources are techniques used to enrich data for ML training. Data labels provide crucial information about datasets, from which an AI algorithm uses to learn.

So, it is imperative that the information should be correct and consistent. We aggregated and used SAD as an additional training set along with PAED. For the labeling strategy, we implemented a labeling OSA algorithm to pre-process SAD to match the standard OSA annotation from PAED based on the AASM guidelines [47], where Apnea events should be labeled with one-minute interval.

2.3.2. Improving ECG Pre-processing and Feature Engineering

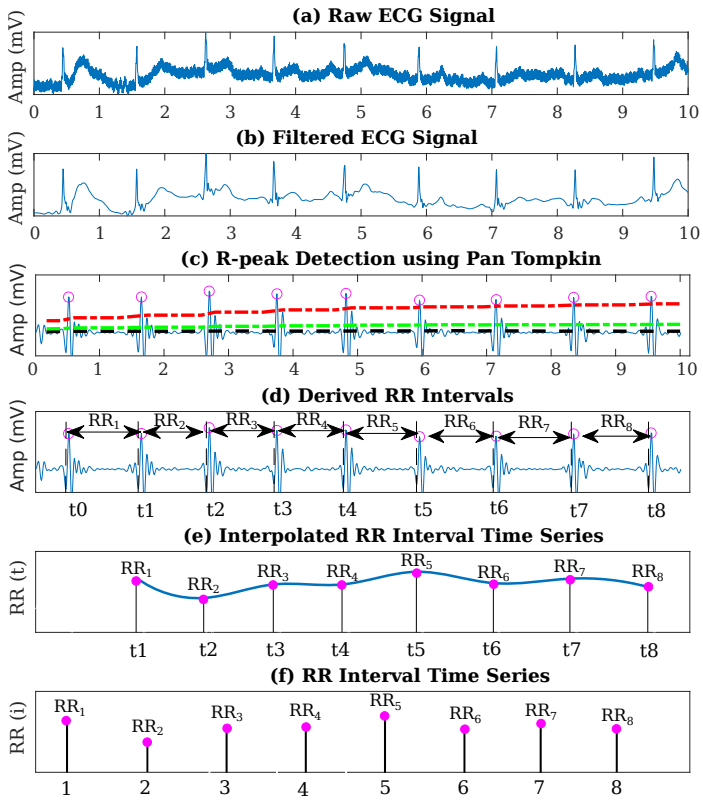


Figure 4: Overview of the pre-processing procedure and data cleaning on extracted features. One single-lead ECG signal was filtered by a wavelet transform method and removed noise. Then RR-intervals time series was extracted and interpolated for model input.

Data preprocessing transforms raw data into the right quality pattern for training. There are various data preprocessing strategies, including data cleaning, balancing, replacing, imputing, partitioning, scaling, augmenting, and unbiasing. We implemented data cleaning and partitioning to process

the raw ECG data. The raw data are often noisy and incomplete due largely to faulty sensors or communication errors. To mitigate the negative impact of noise and incomplete data on the performance of the proposed system, these data instances are usually discarded or interpolated. Fig. 4 shows our pre-processing scheme in order to extract the right input RR intervals and R-peak amplitudes for the proposed model. QRS complexes or R-peaks can be detected with different algorithms such as Hilbert transform [48], Pan and Tompkins [49], and pattern recognition [50]. Problems may appear, especially with ECG signals containing artifacts, irregular rhythms, or frequently varying morphology of the QRS complexes. The more accurate the R-peak detection method, the fewer missed and false R-peaks to be edited in RR intervals [51]. In this study, the raw ECG signal is first denoised by using discrete wavelet transform based on the empirical Bayesian method with a Cauchy prior [52] and then removing unexpected data points, as Fig. 4(b). After that, the position of R-peaks can be accurately determined by using the Pan-Tompkins algorithm, as shown in Fig. 4(c). After R-peaks have been detected, a RR interval time series can be derived referring to the interval between successive ECG R-wave occurrence times as Fig. 4(d). The RR interval is obtained as the difference between the R-wave occurrence times $RR_n = t_n - t_{n-1}$ [see Fig. 4(e)]. Additionally, a median filter is adopted to eliminate unexpected data spikes [10]. The time series constructed from all available RR intervals is not equidistantly sampled but has to be presented as a function of time, i.e., values (t_n, RR_n) [see Fig. 4(f)]. To satisfy this requirement, the cubic spline interpolation [53] has been utilized to resample RR interval time series. The smooth curves are estimated through a number of interpolated data points by fitting a third-degree polynomial. The interpolated RR intervals are equally spaced. The detected R-peaks were used to compute R-peak amplitudes, and the same procedure as creating RR interval time series was applied for the computed R-peak amplitudes. After interpolation, the handcrafted features can be used for further analysis. Next, feature engineering is a process of selecting and transforming variables that will be used as the input for machine learning or statistical prediction models. Feature engineering typically includes feature creation, feature transformation, feature extraction, and feature selection, as listed in Fig. 3. Thus far, we have implemented the feature selection and feature transformation. The use of handcrafted features is described in the model-centric approach section.

2.3.3. Handling False RR Intervals

Data removal and interpolation are two main methods to address the false RR interval. The abnormal RR intervals are removed, and the preceding normal RR intervals are then shifted to replace the deleted ones. Deletion editing decreases the length of RR intervals that may have a significant influence on heart rate variability. In addition, if all the segments containing ectopic beats or other artifacts are deleted in an attempt to eliminate any possible interference in HRV analysis, this can lead to an unacceptable and systematic loss of information. Unlike the deletion methods, interpolation methods replace the non-normal RR intervals with new interpolated RR intervals, preserving the initial number of samples. The interpolation output is shown in the Fig. 4 (e).

2.3.4. Data Segmentation and Apnea Label Sequence

The proposed model takes a sequence of RR-interval, and R-peak amplitude records as its input. These sequences are segmented into one-minute intervals. This is based on the findings of [54], which examined different epoch lengths. It was determined that segments of $60s$ provide the best performance when discriminating between respiratory events. For each patient p, we can divide his/her pulse sequence into n_p one-minute intervals. Then, for every second in the *i-th* interval ($1 \leqslant i \leqslant n_p$), we can obtain one single RR-interval record as follows

$$I(t) = t(R_n) - t(R_{n-1}),$$

where t returns the timestamp function of a pulse R, and R_n and R_{n-1} are two consecutive pulses around the time t. For example, obtaining a 60-dimensional feature vector, denoted by $r_i = [I(1), I(2), ..., I(60)]$. Ultimately, by stacking all n_p feature vectors, we can represent the RR-interval sequence for patient p using a feature matrix $\tilde{R}_p = \{r_1, r_2, ..., r_{n_p}\}$. Note that in the rest of this study, each one-minute interval r_i will be referred to as a detection epoch. A binary label $b_i \in \{0, 1\}$ is used to denote whether there is an apnea in the *i-th* epoch, where 1 is for positive and 0 is for negative. Then we can get apnea label sequence $Y_p = \{b_1, b_2, ..., b_{n_p}\}$.

2.3.5. Data Partition

Based on the segment formulation for patient p, the PAED dataset consists of 70 recordings ($p = 70$), then a total of 33,976 epochs were calculated;

of these epochs, 12,966 were annotated as "apnea", and the remaining 21,010 were annotated as "normal". The SAD dataset has 1,983 epochs with 345 epochs labeled as "apnea". Then the dataset was divided into two sets. The training set was used for model construction and parameter estimation with cross-validation and hyper-parameter tuning. The test set was used to assess the final model on the unseen data.

2.4. Model-centric Approach

We review the basic concept of an LSTM cell and explain the customized stacked LSTM architecture in the context of OSA detection. The notation of this session is shown in Table. 2. The illustration of the system is in the following subsections.

2.4.1. A Basic LSTM Cell

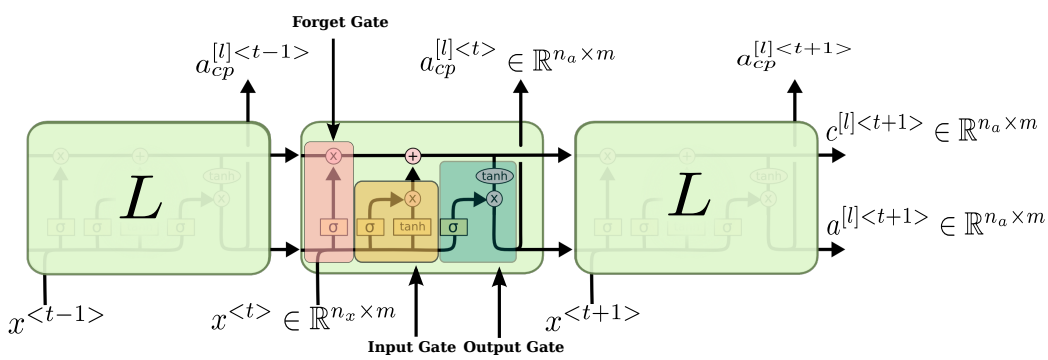


Figure 5: A graphical illustration of a basic LSTM cell. The three gates-forget, input and output- control the interactions between the cell state c and the hidden state a.

LSTM networks are a special type of recurrent neural networks, capable of learning long-term dependencies between timesteps of sequence data while being resilient to the vanishing gradient problem [55]. An LSTM cell has two recurrent features, denoted by $\boldsymbol{a}$ for the hidden state and $\boldsymbol{c}$ for the cell state. The cell, denoted by L, is a mathematical function that takes three inputs and produces two outputs as follows:

$$(\boldsymbol{a}^{<t>}, \boldsymbol{c}^{<t>}) = L(\boldsymbol{a}^{<t-1>}, \boldsymbol{c}^{<t-1>}, \boldsymbol{x}^{t}) \tag{1}$$

Table 2: Notations for Stacked LSTM.

Notation	Description	Dimension/Range
$[l]$	The lth layer. Example: $\boldsymbol{a}^{[4]}$ is the 4th layer hidden state. $\boldsymbol{W}^{[5]}$ and $\boldsymbol{b}^{[5]}$ are the 5th layer parameters	$[1, 2, 3, \ldots, k]$, where k is the last layer
(i)	The ith example (sequence or epoch). Example: $\boldsymbol{x}^{(i)}$ is the ith training example input	$[0, \ldots, N)$, where N is the total number of examples input
$< t >$	Superscript $< t >$ denotes an object at the tth timestep. Example: $\boldsymbol{x_1}^{<t>}$ is the input $\boldsymbol{x}$ at the tth timestep. $\boldsymbol{x_1}^{(i)<t>}$ is the input at the tth timestep of example i	$[1, ..., T_{\boldsymbol{x_1}}]$
m	The number of examples in a mini-batch (batch size)	$(n_{\boldsymbol{x}}, m, T_{\boldsymbol{x_1}})$
$\boldsymbol{x_1}$	The 3D input vector including RR intervals and R-peak amplitudes is fed into the first LSTM layer	$(n_{\boldsymbol{x_1}}, m, T_{\boldsymbol{x_1}})$
$n_{\boldsymbol{x_1}}$	The number of features of input $\boldsymbol{x_1}$ (i.e., length of input vector $\boldsymbol{x_1}$)	2
$T_{\boldsymbol{x_1}}$	The number of timesteps of input sequence. A single training example $\boldsymbol{x_1}^{(i)}$ consists of multiple time steps $T_{\boldsymbol{x_1}}$	
$\boldsymbol{x_1}^{<t>}$	The input data at the time step t	$(n_{\boldsymbol{x_1}}, m)$
$\boldsymbol{x_1}^{(i)<t>}$	A single training input vector associated with the ith example (sequence) at the time step t	$(n_{\boldsymbol{x_1}}, 1)$
$\boldsymbol{x_2}$	The 2D input vector represents the demographic data	$(n_{\boldsymbol{x_2}}, m)$
$n_{\boldsymbol{x_2}}$	The number of features (i.e., length) of input vector $\boldsymbol{x_2}$	2
$\boldsymbol{a}$	The 3D vector of hidden state of a LSTM cell	$(n_{\boldsymbol{a}}, m, T_{\boldsymbol{x_1}})$
$n_{\boldsymbol{a}}$	The length of a hidden state (i.e hidden unit dimension)	
$\boldsymbol{a}^{[l]<t>}$	The hidden state at time step t at the layer l. Example: $\boldsymbol{a}^{[3]<4>}$ is the hidden state of the 3rd layer and the 4th time step. $\boldsymbol{a}^{[k]<T_{\boldsymbol{x_1}}>}$ is the last hidden state of the last layer k	$(n_{\boldsymbol{a},m})$
$\boldsymbol{\Gamma}_f$	The forget gate	$[0; 1]$
$\boldsymbol{W}_f$	Weight matrix of the forget gate	$(n_{\boldsymbol{a}}, n_{\boldsymbol{a}} + n_{\boldsymbol{x_1}})$
$\boldsymbol{b}_f$	Bias of the forget gate	$(n_{\boldsymbol{a}}, 1)$
$\boldsymbol{\Gamma}_u$	The input gate	$[0, 1]$
$\boldsymbol{W}_u$	Weight matrix of the update gate	$(n_{\boldsymbol{a}}, n_{\boldsymbol{a}} + n_{\boldsymbol{x_1}})$
$\boldsymbol{b}_u$	Bias of the update gate	$(n_{\boldsymbol{a}}, 1)$
$\boldsymbol{\Gamma}_o$	The output gate	$[0, 1]$
$\boldsymbol{W}_o$	Weight matrix of the output gate	$(n_{\boldsymbol{a}}, n_{\boldsymbol{a}} + n_{\boldsymbol{x_1}})$
$\boldsymbol{b}_o$	Bias of the output gate	$(n_{\boldsymbol{a}}, 1)$
$\boldsymbol{c}$	The memory cell state including all time steps	$(n_{\boldsymbol{a}}, m, T_{\boldsymbol{x_1}})$
$\boldsymbol{W}_c$	Weight matrix of the memory cell state $\boldsymbol{c}$	$(n_{\boldsymbol{a}}, n_{\boldsymbol{a}} + n_{\boldsymbol{x_1}})$
$\boldsymbol{c}^{[l]<t>}$	The cell state at the time step t and the layer l is the "memory" that gets passed onto future time steps	$(n_{\boldsymbol{a}}, m)$
$\boldsymbol{c}^{[l]<t-1>}$	The previous cell state $\boldsymbol{c}^{<t-1>}$ at the layer l and the time step $t-1$ is adjusted (weighted) by the forget gate $\boldsymbol{\Gamma}_f^t$	$(n_{\boldsymbol{a}}, m)$
$\tilde{\boldsymbol{c}}^{[l]<t>}$	The candidate value $\tilde{\boldsymbol{c}}$, adjusted by the update gate $\boldsymbol{\Gamma}_u^{<t>}$	
$\hat{\boldsymbol{y}}$	Detection output (i.e., OSA or non-OSA)	$[0; 1]$

Both outputs leave the cell at time step t and are fed back into that same cell at that time point $t+1$. At any time step t, an element of the input sequence $\boldsymbol{x}^t$ is also fed into the cell, as shown in Fig. 5.

Inside the cell, only one specific layer $[l]$ is considered for notations to explain how an LSTM cell works. Let us assume we are reading the sequences in a piece of RRI and R-peak amplitude, and plan to use an LSTM to keep track of sleep apnea events, such as whether a window sequence of RRI has OSA events (episode) or not. If the characteristic of input sequences changes, we need to find a way to get rid of our previously stored memory value of the OSA/non-OSA state (i.e., the memory of the previous state becomes outdated), so we will "forget" that outdated state by using the forget gate equation:

$$\boldsymbol{\Gamma}_f^{<t>} = \sigma(\boldsymbol{W}_f[\boldsymbol{a}^{<t-1>}, \boldsymbol{x}^{<t>}] + \boldsymbol{b}_f), \tag{2}$$

where $\boldsymbol{W}_f$ contains weights that govern the forget gate's behavior. The previous time step's hidden state $\boldsymbol{a}^{<t-1>}$ and the current time step's input $\boldsymbol{x}^{<t>}$ are concatenated together and multiplied by $\boldsymbol{W}_f$. A sigmoid function is used to squish each of the gate tensor's values $\boldsymbol{\Gamma}_f^{<t>}$ range from 0 to 1. This is helpful to update data because any number getting multiplied by 0 is 1, causing values to disappear or be "forgotten", and any number multiplied by 1 is the same value; therefore, that value stays the same or is "kept". The network can learn which data is not important therefore can be forgotten or which data is important to keep. In addition, the forget gate $\boldsymbol{\Gamma}_f^{<t>}$ has the same dimensions as the previous cell state $\boldsymbol{c}^{<t-1>}$. This means that the two can be element-wise-multiplied together. Multiplying the tensors $\boldsymbol{\Gamma}_f^{<t>} * \boldsymbol{c}^{<t-1>}$ is like applying a mask over the previous cell state. If a single value in $\boldsymbol{\Gamma}_f^{<t>}$ is 0 or close to 0, then the product is close to 0. This keeps the information stored in the corresponding unit in $\boldsymbol{c}^{<t-1>}$ from being remembered for the next time step. Similarly, if one value is close to 1, the product is close to the original value in the previous cell state. The LSTM will keep the information from the corresponding unit of $\boldsymbol{c}^{<t-1>}$, to be used in the next time step.

The candidate value $\tilde{\boldsymbol{c}}^{<t-1>}$ is a tensor containing information from the current time step that may be stored in the current cell state $\boldsymbol{c}^{<t>}$. The parts of the candidate value that get passed on depend on the update gate $\boldsymbol{\Gamma}_u$, which is described below. The candidate value is a tensor containing values that range from -1 to 1 produced by the tanh function. When vectors are flowing through the LSTM network, they undergo many transformations due to various math operations, then some values can be large and become

astronomical, causing other values to seem insignificant. So, the purpose of the tanh function is to help regulate the network.

$$\tilde{\boldsymbol{c}}^{<t>} = \tanh(\boldsymbol{W}_c[a^{<t-1>}, x^{<t>}] + b_c) \tag{3}$$

The second gate in the LSTM cell that is update gate $\boldsymbol{\Gamma}_u$, as described in Equation 4. The gate decides what aspects of candidate $\tilde{\boldsymbol{c}}^{<t>}$ to add to the cell state $\boldsymbol{c}^{<t>}$. The update gate decides what parts of a "candidate" tensor $\tilde{\boldsymbol{c}}^{<t>}$ are passed onto the cell state $\boldsymbol{c}^{<t>}$. The update gate is a tensor containing values between 0 and 1, produced by the sigmoid function. When a unit in the update gate is close to 1, it allows the value of the candidate $\tilde{\boldsymbol{c}}^{<t>}$ to be passed onto the hidden state $\boldsymbol{c}^{<t>}$. When a unit in the update gate is close to 0, it prevents the corresponding value in the candidate from being passed onto the hidden state. The update gate is multiplied element-wise with the candidate, and this product $\boldsymbol{\Gamma}_f^{<t>} * \tilde{\boldsymbol{c}}^{<t>}$ is used in determining the cell state $\boldsymbol{c}^{<t>}$.

$$\boldsymbol{\Gamma}_u^{<t>} = \sigma(\boldsymbol{W}_f[\boldsymbol{a}^{<t-1>}, \boldsymbol{x}^{<t>}] + \boldsymbol{b}_u). \tag{4}$$

The new cell state is a combination of the previous cell state $\boldsymbol{c}^{<t-1>}$ and the candidate value $\tilde{\boldsymbol{c}}^{<t>}$:

$$\boldsymbol{c}^{<t>} = \boldsymbol{\Gamma}_f^{<t>} * \boldsymbol{c}^{<t-1>} + \boldsymbol{\Gamma}_u^{<t>} * \tilde{\boldsymbol{c}}^{<t>}. \tag{5}$$

Finally, the output gate $\boldsymbol{\Gamma}_o$ determines the value of the next hidden state. First, we pass the previous hidden state and the current input into a sigmoid function. The output gate is like the other gates, containing values ranging from 0 to 1.

$$\boldsymbol{\Gamma}_o^{<t>} = \sigma(\boldsymbol{W}_o[a^{<t-1>}, \boldsymbol{x}^{<t>}] + \boldsymbol{b}_o). \tag{6}$$

Then we pass the newly modified cell state to the tanh function. We multiply the tanh output with the sigmoid output to decide what information the hidden state should carry.

$$\boldsymbol{a}^{<t>} = \boldsymbol{\Gamma}_o^{<t>} * \tanh(\boldsymbol{c}^{<t>}). \tag{7}$$

The output of the current cell is the hidden state of the next cell. The new cell state and the new hidden state is then carried over to the next time step. Notice that the dimensions of the weight matrices $\boldsymbol{W}$ for all gates are entirely determined by the hidden state size n_a and the number of input variables

$\boldsymbol{x}_1$, which makes sense. In reality, we are processing a huge bunch of data, so we will rarely be running time-series data samples through the LSTM model one at a time. Rather, we will be processing them in batches, so there is an added parameter of batch size m. The basic LSTM cell L is presented in Algorithm 1.

Algorithm 1: The *basic_LSTM_Cell* algorithm

Input: $\boldsymbol{x}^{<t>} \in \mathbb{R}^{n_x \times m}$; $\boldsymbol{a}^{[l]<t-1>}, \boldsymbol{c}^{[l]<t-1>} \in \mathbb{R}^{n_a \times m}$;
$\boldsymbol{W}_f, \boldsymbol{W}_u, \boldsymbol{W}_c, \boldsymbol{W}_o \in \mathbb{R}^{n_a \times (n_a + n_x)}$; $\boldsymbol{b}_f, \boldsymbol{b}_i, \boldsymbol{b}_c, \boldsymbol{b}_o, \boldsymbol{b}_y \in \mathbb{R}^{n_a \times 1}$

Output: next hidden state: $\boldsymbol{a}^{[l]<t>} \in \mathbb{R}^{n_a \times m}$; next memory state: $\boldsymbol{c}^{[l]<t>} \in \mathbb{R}^{n_a \times m}$; next cell input at layer $l+1$: $\boldsymbol{a}_{cp}^{[l]<t>} \in \mathbb{R}^{n_a \times m}$

1 Concatenate $\boldsymbol{a}^{[l]<t-1>}$ and $\boldsymbol{x}^{<t>}$ in a single matrix:

$$temp = \begin{bmatrix} \boldsymbol{a}^{[l]<t-1>} \\ \boldsymbol{x}^{<t>} \end{bmatrix};$$

2 Compute values for $\boldsymbol{\Gamma}_f^{<t>}$, $\boldsymbol{\Gamma}_u^{<t>}$, $\boldsymbol{\Gamma}_o^{<t>}$, $\tilde{\boldsymbol{c}}^{<t>}$, $\boldsymbol{c}^{<t>}$ and $\boldsymbol{a}^{<t>}$:

$$\boldsymbol{\Gamma}_f^{<t>} = \sigma(\boldsymbol{W}_f * temp + \boldsymbol{b}_c);$$

$$\tilde{\boldsymbol{c}}^{<t>} = \tanh(\boldsymbol{W}_c * temp + \boldsymbol{b}_f);$$

$$\boldsymbol{\Gamma}_u^{<t>} = \sigma(\boldsymbol{W}_f * temp + \boldsymbol{b}_u);$$

$$\boldsymbol{c}^{<t>} = \boldsymbol{\Gamma}_f * \boldsymbol{c}^{<t-1>} + \boldsymbol{\Gamma}_u^{<t>} * \tilde{\boldsymbol{c}}^{<t>});$$

$$\boldsymbol{\Gamma}_o^{<t>} = \sigma(\boldsymbol{W}_o * temp + \boldsymbol{b}_o);$$

$$\boldsymbol{a}^{<t>} = \boldsymbol{\Gamma}_o * \tanh(\boldsymbol{c}^{<t>});$$

3 Duplicate the next hidden state $\boldsymbol{a}^{[l]<t>}$ as an input for the next LSTM layer l:

$$\boldsymbol{a}_{cp}^{[l]<t>} = \boldsymbol{a}^{[l]<t>};$$

Note: Dimension of $\boldsymbol{x}^{<t>}$ can be $(n_x \times m)$ at the first input layer or $(n_a \times m)$ at next LSTM layers l.

2.4.2. Customized Stacked LSTM Architecture

Motivated by LSTM networks' capability of learning long-term dependencies between timesteps of sequence data [55], we introduce in this section an

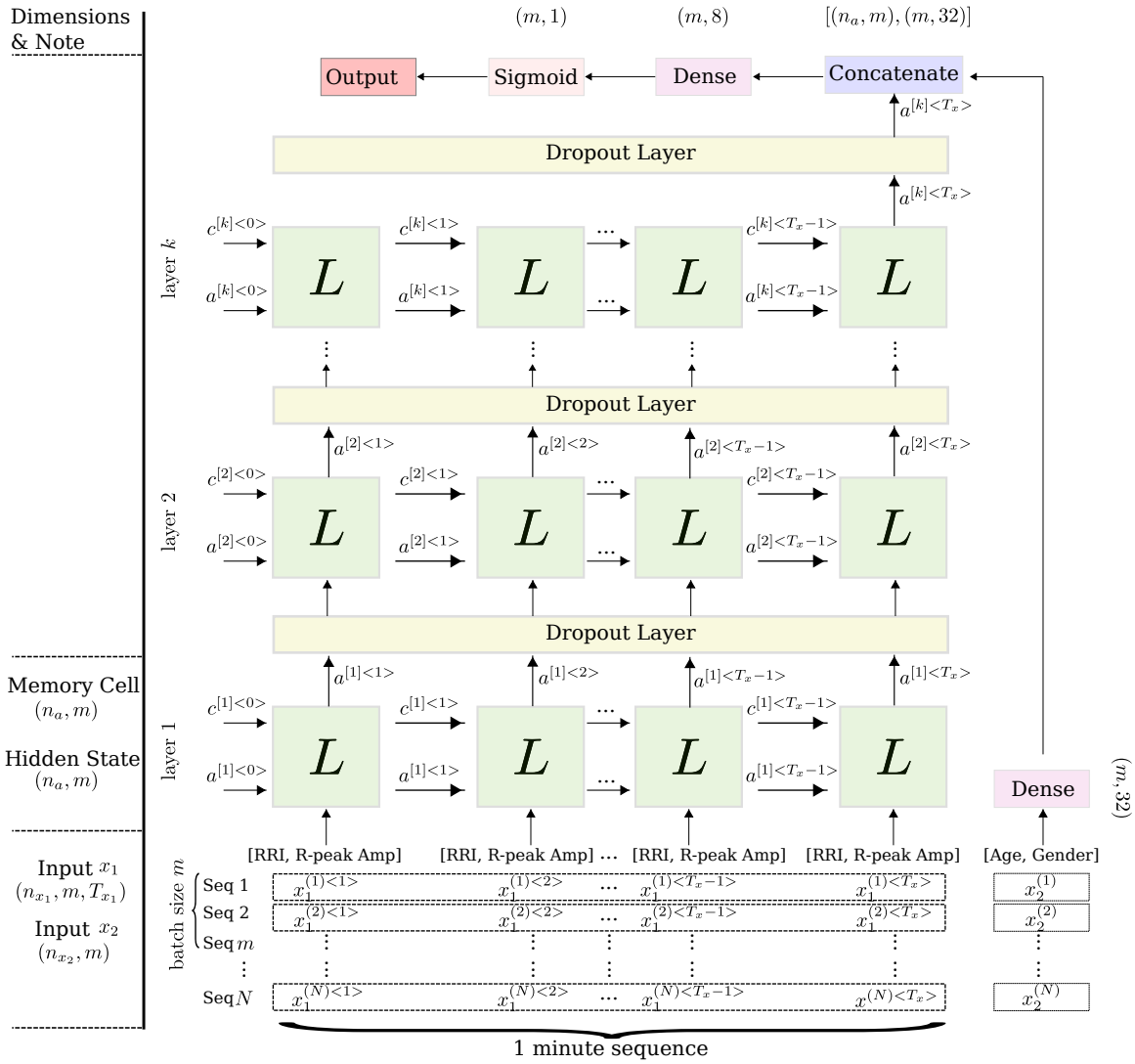


Figure 6: The proposed stacked LSTM architecture for OSA detection. The customized network consists of the input layer, the stack LSTM layers and the output. RR Intervals and R-peak amplitudes, and demographics such as age and gender are represented as vectors $\boldsymbol{x}_1$, and $\boldsymbol{x}2$, respectively. The input features are fed into the first LSTM layer. The memory cell L contains the cell state $\boldsymbol{c}$ ensuring the gradient can pass across many time steps without vanishing or exploding. The hidden state $\boldsymbol{a}$ is the output of a cell, which is updated through stacked LSTM layers to maximize the learning insights. The final hidden state will be used to make a prediction.

Algorithm 2: The customized stacked LSTM-based model for OSA detection algorithm

Input: $\boldsymbol{x}_1 \in \mathbb{R}^{n_{x_1} \times m \times T_{x_1}}; \boldsymbol{x}_2 \in \mathbb{R}^{n_{x_2} \times m}$; the layer $l \in \mathbb{Z}^{1,\ldots,k}$; the length of hidden state n_a; and the number of timesteps T_{x_1}

Output: $\hat{y} \in \mathbb{Z}^{0,1}$

Initialize $l; m; n_a; T_{\boldsymbol{x}_1}$;
while $l \leqslant k$ **do**
  Initialize $\boldsymbol{a}^{[l]<0>} = 0; \boldsymbol{c}^{[l]<0>} = 0; t = 1$;
  while $t \leqslant T_{x_1}$ **do**
    $\boldsymbol{a}^{[l]<t>}, \boldsymbol{c}^{[l]<t>}, \boldsymbol{a}_{cp}^{[l]<t>} =$
    $basic_LSTM_Cell(\boldsymbol{x}_1, \boldsymbol{a}^{[l]<0>}, \boldsymbol{c}^{[l]<0>}, \boldsymbol{W}, \boldsymbol{b})$;
  Reduce overfitting by using dropout;
  if $l \neq k$ **then**
    Update $\boldsymbol{x}_1 = \boldsymbol{a}_{cp}^{[l]}$ as an input for the next LSTM layer;
  else
    Get the last output cell from last LSTM layer
    $\boldsymbol{a}^{[k]<T_{\boldsymbol{x}1}>} = \boldsymbol{a}_{cp}^{[k]<T_{\boldsymbol{x}1}>}$;
Concatenate $\boldsymbol{a}^{[k]<T_{\boldsymbol{x}_1}>}$ and $\boldsymbol{x}_2$

$$merged = [(n_a, m), (m, 32)];$$

Train the model with $\boldsymbol{x}_1, \boldsymbol{x}_2$ and update weights with backward steps;
Detect OSA or non-OSA using *sigmoid* activation function

$$\hat{y} = sigmoid(\boldsymbol{W}_y * \boldsymbol{a} + \boldsymbol{b}_y);$$

end-to-end model for OSA detection using handcrafted ECG features. The proposed approach leverages the stacked learning paradigm. Existing studies [56, 57] have shown that deep LSTM architectures with several hidden layers can build up a progressively higher level of representations of sequence data and, thus, work more effectively. This stacked-layers mechanism, which can enhance the power of neural networks, is adopted in this study. Fig. 6 represents the overall system model of the proposed deep neural network for OSA detection. There are two groups of input data (i.e., RR-Interval, R-peak amplitude time series, and demographic data) that are fed into the model. The architecture comprises three LSTM layers. Specifically, all timesteps T_{x_1} get put through the first LSTM layer to generate a set of hidden states (one per timestep). These hidden states $a^{k<T_x>}$ are then used as inputs for the second LSTM layer to generate another set of hidden states, and so on and so forth. The entire series of hidden states $a^{k<T_x>}$ to feed forward into each successive LSTM layer. The last layer of the LSTM outputs, a vector $a^{[k]<T_x>}$, fed as the input of a fully connected network (i.e., dense layer) to transform the hidden state into a binary classification by using a sigmoid function, where the output either apnea or non-apnea is determined. A dropout layer is added after each LSTM layer to reduce over-fitting and improve the generalization of the model. The customized stacked LSTM procedure is shown in Algorithm 2.

3. Experimental Results

3.1. Implementation and Hyper-parameters Settings

Native Tensorflow with wrapper Keras implementations was used for implementing the stacked LSTM architecture. All experiments were run on NVIDIA GeForce RTX 3070 GPUs with CUDA 11.4 on a machine with an Intel Xeon 3.7 GHz with 18 cores, and 128GB RAM.

Estimating what hyper-parameters to use to fit the complexity of input data is the main course in any deep learning task. There are several rules of thumb that increase either types of complexity hidden state size n_a, dropout at each layer and the number of LSTM layers k. Generally, the input variables RR-interval and R-peak amplitude time-series data are interdependent. A bigger hidden state size would be necessary to capture the structural and semantic interdependences of the input feature sequence. Similarly, the patterns in our time-series data are abstracted, and a greater model depth (i.e., number of hidden layers) is necessary. Practically, n_a is not a small number. Since the model aims to classify the input into two categories: OSA and

no-OSA events, it is a two-class logistic regression problem. Thus, we prefer to adopt a sigmoid function rather than a softmax function that is used for a multiclass problem. Advanced adaptive learning rate methods, such as RMSprop, Adagrad, and Adam, are also applied to reduce the effect of the initial and potentially sub-optimal learning rate for model training. Based on the preliminary experiments, the hyper-parameters, such as a standard stochastic gradient descent optimizer, a learning rate of $1E-2$, and a batch size of 128, are appropriate for our models. To reduce overfitting, we adopt the most common regularization technique – dropout with a dropout rate of 20%. Another popular method we employed is "early stopping", which means the training process will be stopped automatically at the best point. Thus, the latest parameters that perform best on the validation set are chosen as soon as the training process stops.

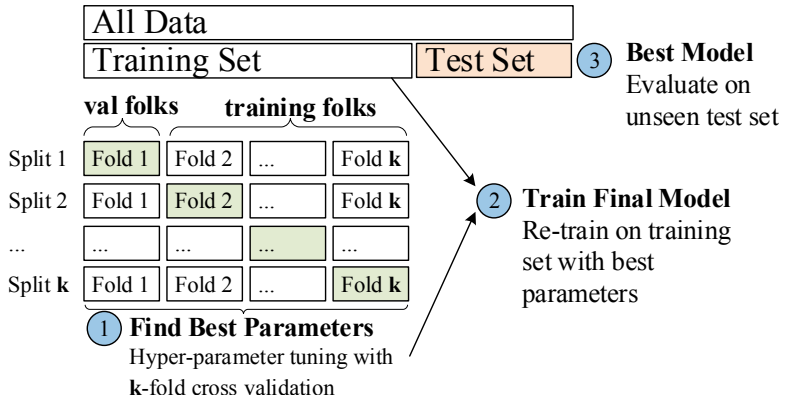


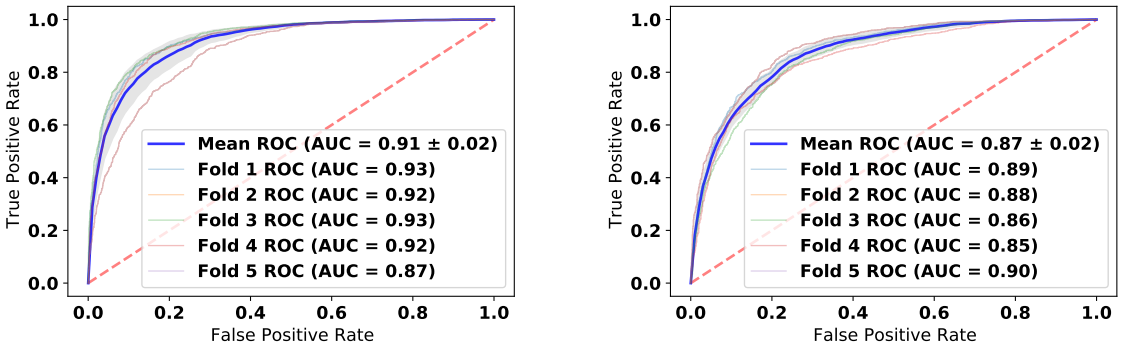
Figure 7: Hyper-parameter tuning process with k-fold cross validation.

Before performing k-fold cross-validation, the dataset should be shuffled to avoid bias. The strategy for hyper-parameter tuning is shown in Fig. 7. First, (1) we find the best hyper-parameter sets with k-fold cross-validation. Random search for hyper-parameter optimization was implemented to find the best hyper-parameter sets [58], which randomly feeds different combinations of desired hyper-parameters through the algorithm and then reports the best performing combination. Then, (2) we re-train the model with the best parameters on the training set. The reason behind fitting the best model to

the whole training set after k-fold cross-validation is to provide more training samples to the learning algorithm of the best model. This usually results in a more accurate and robust model. Finally, (3) the model is evaluated with an unseen test set.

3.2. Results of Learning Performance

3.2.1. Performance with/without data-centric



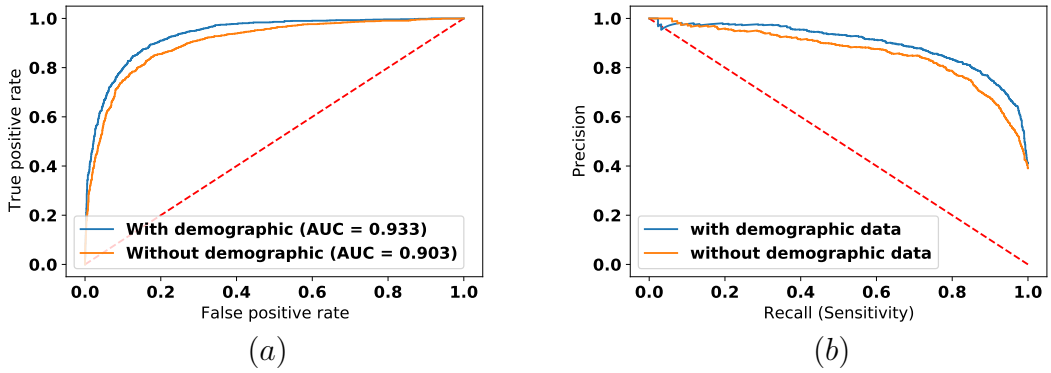
(*a*) (*b*)

Figure 8: Comparison of Receiver Operating Characteristic (ROC) curves using 5-fold cross-validation scheme for the customized stack LSTM model to detect OSA events with data-centric approach (a) and without data-centric approach (b). The ROC curve showed the change of the TPR (Sensitivity) value with FPR (1 - Specificity) value under different thresholds. AUC-ROC is maximized in the upper left corner.

We performed 5-fold cross-validation to evaluate the model performance after implementing data-centric strategies. Fig. 8 (a) shows that the ROC curve hugs the upper left corner of the graph, indicating that the model performance with the data-centric approach is improved at distinguishing OSA and non-OSA events than one without implementing data-centric methods. To quantify this, the AUC score was calculated (AUC = 0.91, and 0.87 for Fig. 8 (a), and Fig. 8 (b), respectively).

3.2.2. Performance with Customized Stacked LSTM

To evaluate the performance of the proposed model, we measured the ROC curves and also tested them with the Precision-Recall curve, which is useful when there is a moderate class imbalance in our datasets. Fig. 9



(a) (b)

Figure 9: ROC and Precision-Recall curves of the detection performance with/without demographic data. (a) ROC curves. (b) Precision-Recall curves.

(a) shows that the ROC curve (AUC score is 0.935) with demographic data as an additional input consistently outperforms that without demographic data. Moreover, Fig. 9 (b) shows that the model is robust even when there is an imbalance between OSA and non-OSA classes in the training set. As a result, stacking LSTM hidden layers and feeding more relevant features as demographic data make the model more efficient and better extracting the description from input data.

3.3. Model Generalization on SAD Cancer Data

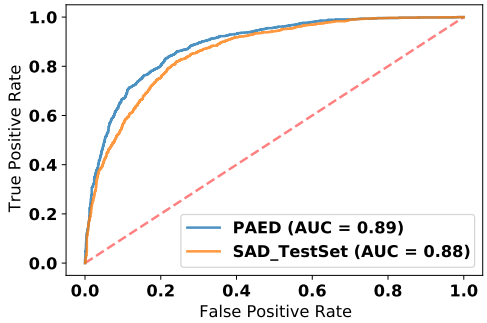


Figure 10: ROC curve and AUC score calculated using SAD as a test set.

We used SAD as the test set to evaluate the generalization of the model on

cancer patients' data with different characteristics compared to PAED (no-cancer patients). The result in Fig. 10 indicates that our proposed model is able to perform well even on cancer patients (AUC = 0.89, and 0.88, respectively). Although the sleep patterns of these cancer patients are different from those of healthy subjects (PEAD), the proposed method can still achieve high accuracy. This demonstrates that the proposed hybrid approach has broader applicability and higher effectiveness on unseen datasets. If the training set contained a certain number of cancer subjects, the recognition performance would be much better, as shown by the result in Fig. 8 (a).

3.4. Effects of Hyper-parameters

3.4.1. The Effect of Learning Rate

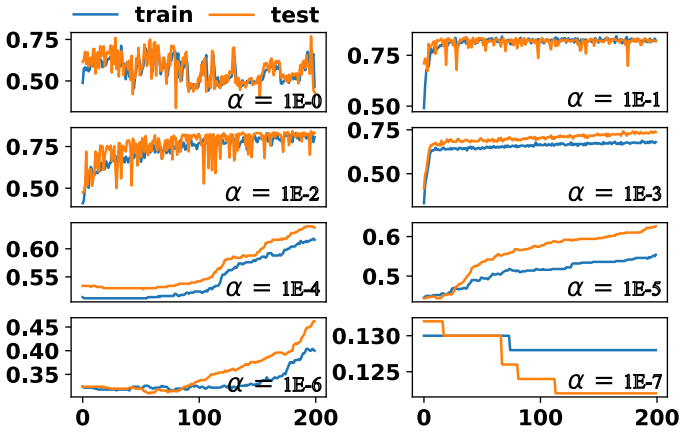


Figure 11: Model accuracy on train and test sets with different learning rates α.

We assessed learning performance on a logarithmic scale from $1E-0$ to $1E-7$. Fig. 11 shows oscillations in behavior at a too-large learning rate of $1E-0$ and the inability of the model to learn anything at too-small learning rates of $1E-6$ and $1E-7$. However, at learning rates of $1E-1$, $1E-2$, and $1E-3$, the model is able to perform well, the accuracy curves converge, even though it slows down as the learning rate decreases. With the chosen model configuration, the results indicate that a moderate learning rate of $1E-1$ resulted in good and stable model performance on the training and test sets.

3.4.2. The Effect of Optimizers

Fig. 12 shows that with the default $\alpha = 1E-1$, SGD does learn the problem but takes nearly all 200 epochs and leads to fluctuating accuracy

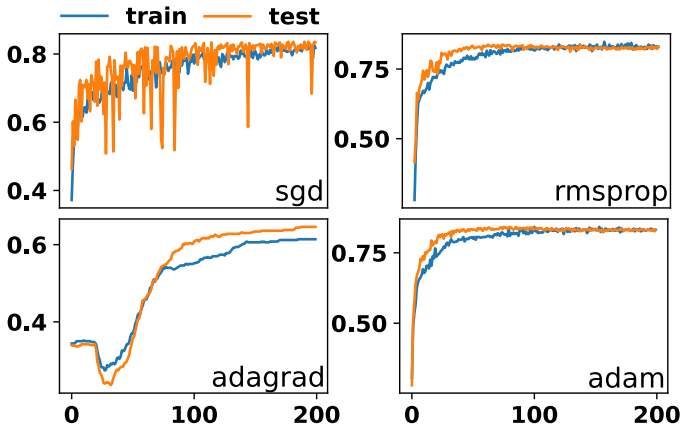


Figure 12: Model accuracy on train and test sets with different optimizers at $\alpha = 1E - 1$. Four optimizers include Stochastic Gradient Descent (SGD), Root Mean Squared Propagation (RMSProp), Adaptive Gradient Algorithm (Adagrad), and Adaptive Moment Estimation (Adam).

on the training data and much more on the test set. The plots show that all three adaptive learning rate methods learn the problem faster and with dramatically less volatility in training and test set accuracy. Both RMSProp and Adam demonstrate similar performance, effectively learning the problem within 50 training epochs and spending the remaining training time making very minor weight updates.

3.5. Compare to Baselines

Table 3: Overall performance of the model compared to baseline methods.

Methods	**AUC**
SVM	0.71
RNN	0.83
GRU	0.87
LSTM	0.89
CNN-LSTM	0.90
Proposed Hybrid	**0.93**

To evaluate the robustness of the system, we compared our proposed hybrid approach for OSA detection with SVM, RNN, GRU, LSTM, and CNN-LSTM methods with basic feature pre-processing, and without model

and data quality tuning using the same PAED dataset. The result is shown in Table. 3. It can be observed that the performance of the proposed method in ROC score is better than that of the conventional methods.

4. Discussion

In this session, we discussed further aspects of data-centric strategies including the balance between data quality and challenges in feature engineering in order to fullly utilize the advantages of the hybrid approach.

4.1. What is the right balance to achieve success?

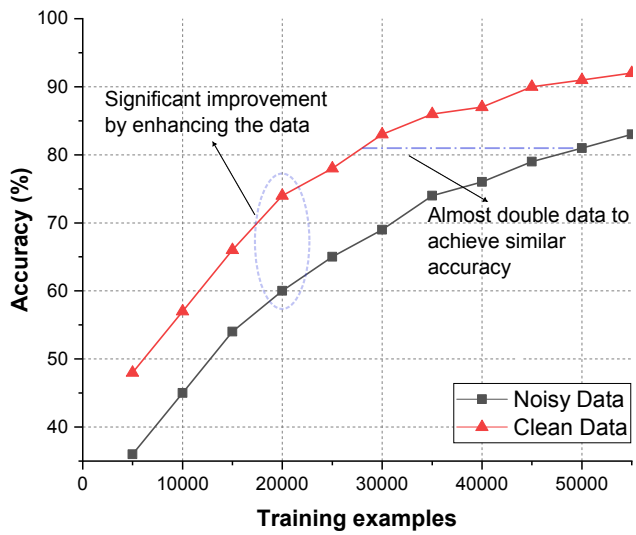


Figure 13: The effects of data quality on learning performance.

We often try to find the right amount of data to build meaningful models. The quality of the data takes precedence over the quantity of the data. One can have millions of rows in a dataset that contains many noisy observations that interfere with the learning process. On the other hand, a smaller dataset with higher-quality labels often leads to much better results. As demonstrated in Fig. 13, with the same $20,000$ training examples, the model with clean data achieved around 75% accuracy, whereas the model with noisy data achieved only 60% accuracy. In addition, we used fewer training examples, around $28,000$ compared to $50,000$, but still obtained the same 80% accuracy. We can either improve our code or improve our data or

do both. The reality is that the balance between improving code and data is depended on the business goal and the available tooling in the ML process.

4.2. Feature Engineering

Feature engineering is about representing raw data in terms of the most relevant variables (i.e., features) using prior knowledge or an algorithm. The challenge here is to use the domain expertise to extract important features (RR intervals, R-peak amplitudes, and demographics) to improve the quality of a model, rather than providing raw ECG data to the model. For certain data, such as medical data, a collaboration between ML engineers, researchers, and clinical experts is required to accomplish the task of labeling, extracting features, and tuning data.

4.3. Limitations

Implementing the proposed hybrid system, in reality, is a complex work between the deployment of ML applications and human-in-the-loop, where human engineers might make mistakes, need guidance, or need to be warned when something unexpected happens. The study is not yet able to handle the entire process automatically, and it is an open research question. Researchers have started tackling this issue by developing a new technique called MLOps that offers principled capabilities for life-cycle management, monitoring, and validation.

5. Conclusions and future works

In this work, a hybrid data-centric and model-centric approach for OSA diagnosis has been presented. The proposed data-centric strategies improve data quality by correcting data labels, removing noisy data instances, applying feature engineering and domain knowledge, while model-centric AI based stacked LSTM attempts to engineer performance gains by tuning model parameters. This system architecture is considered a prerequisite for our larger picture with federated learning in developing a future trusted and secured at-home monitoring system.

Acknowledgment

This research is supported by National Institute of General Medical Sciences of the National Institutes of Health under Award Number U54GM128729.

References

[1] A. Sharafkhaneh, N. Giray, P. Richardson, T. Young, M. Hirshkowitz, Association of psychiatric disorders and sleep apnea in a large cohort, Sleep 28 (11) (2005) 1405–1411.

[2] American Sleep Association, Sleep statistics 2021, https://www.sleepassociation.org/sleep-apnea/obstructive-sleep-apnea/, Last accessed on 2012 (2021).

[3] A. V. Benjafield, N. T. Ayas, P. R. Eastwood, R. Heinzer, M. S. M. Ip, M. J. Morrell, C. M. Nunez, S. R. Patel, J.-L. Penzel, P. E. Peppard, S. Sinha, S. Tufik, K. Valentine, A. Malhotra, Estimation of the global prevalence and burden of obstructive sleep apnoea: a literature-based analysis, Lancet Respir. Med. 7 (8) (2019) 687–698.

[4] S. Strausz, T. Kiiskinen, M. Broberg, S. Ruotsalainen, J. Koskela, A. Bachour, FinnGen, A. Palotie, T. Palotie, S. Ripatti, H. M. Ollila, Sleep apnoea is a risk factor for severe covid-19, BMJ Open Respiratory Research 8 (1) (2021). arXiv:https://bmjopenrespres.bmj.com/content/8/1/e000845.full.pdf, doi:10.1136/bmjresp-2020-000845.
URL https://bmjopenrespres.bmj.com/content/8/1/e000845

[5] World Health Organisation, Global surveillance, prevention and control of chronic respiratory diseases: a comprehensive approach, https://www.who.int/gard/publications/GARD%20Book%202007.pdf, Last accessed on 2007 (2007).

[6] A. H. Khandoker, M. Palaniswami, C. K. Karmakar, Support vector machines for automated recognition of obstructive sleep apnea syndrome from ECG recordings, IEEE Trans. Inf. Technol. Biomed. 13 (1) (2009) 37–48.

[7] S. G. Jones, B. A. Riedner, R. F. Smith, F. Ferrarelli, G. Tononi, R. J. Davidson, R. M. Benca, Regional reductions in sleep electroencephalography power in obstructive sleep apnea: a high-density EEG study, Sleep 37 (2) (2014) 399–407.

[8] J. Solà-Soler, J. A. Fiz, J. Morera, R. Jané, Multiclass classification of subjects with sleep apnoea-hypopnoea syndrome through snoring analysis, Med. Eng. Phys. 34 (9) (2012) 1213–1220.

[9] C. Avcı, A. Akbaş, Sleep apnea classification based on respiration signals by using ensemble methods, Biomed. Mater. Eng. 26 Suppl 1 (s1) (2015) S1703–10.

[10] L. Chen, X. Zhang, C. Song, An automatic screening approach for obstructive sleep apnea diagnosis based on single-lead electrocardiogram, IEEE Transactions on Automation Science and Engineering 12 (1) (2015) 106–115. doi:10.1109/TASE.2014.2345667.

[11] K. Li, W. Pan, Y. Li, Q. Jiang, G. Liu, A method to detect sleep apnea based on deep neural network and hidden markov model using single-lead ecg signal, Neurocomputing 294 (2018) 94–101. doi:https://doi.org/10.1016/j.neucom.2018.03.011.
URL `https://www.sciencedirect.com/science/article/pii/S0925231218302972`

[12] C. Varon, A. Caicedo, D. Testelmans, B. Buyse, S. Van Huffel, A novel algorithm for the automatic detection of sleep apnea from Single-Lead ECG, IEEE Trans Biomed Eng 62 (9) (2015) 2269–2278.

[13] Andrew Ng, Dillon Laird, and Lynn He., Data-centric ai competition, `https://https-deeplearning-ai.github.io/data-centric-comp/`, Last accessed on 2021 (2021).

[14] Andrew Ng, Tips for a data centric ai approach, `https://landing.ai/tips-for-a-data-centric-ai-approach`, Last accessed on 2022 (2022).

[15] A. Tsymbal, The problem of concept drift: definitions and related work, Tech. Rep. TCD-CS-2004-15, The University of Dublin, Trinity College, Department of Computer Science, Dublin, Ireland (2004).

[16] N. Sambasivan, S. Kapania, H. Highfill, D. Akrong, P. Paritosh, L. M. Aroyo, "everyone wants to do the model work, not the data work": Data cascades in high-stakes ai, in: Proceedings of the 2021 CHI Conference on Human Factors in Computing Systems, CHI '21,

Association for Computing Machinery, New York, NY, USA, 2021. doi:10.1145/3411764.3445518.
URL https://doi.org/10.1145/3411764.3445518

[17] B. Fatimah, P. Singh, A. Singhal, R. B. Pachori, Detection of apnea events from ecg segments using fourier decomposition method, Biomedical Signal Processing and Control 61 (2020) 102005. doi:https://doi.org/10.1016/j.bspc.2020.102005.
URL https://www.sciencedirect.com/science/article/pii/S1746809420301610

[18] J. V. Marcos, R. Hornero, D. Alvarez, F. Del Campo, M. Aboy, Automated detection of obstructive sleep apnoea syndrome from oxygen saturation recordings using linear discriminant analysis, Med. Biol. Eng. Comput. 48 (9) (2010) 895–902.

[19] J. V. Marcos, R. Hornero, D. Álvarez, F. del Campo, C. Zamarrón, Assessment of four statistical pattern recognition techniques to assist in obstructive sleep apnoea diagnosis from nocturnal oximetry, Medical Engineering & Physics 31 (8) (2009) 971–978. doi:https://doi.org/10.1016/j.medengphy.2009.05.010.
URL https://www.sciencedirect.com/science/article/pii/S1350453309001246

[20] S. M. Isa, M. I. Fanany, W. Jatmiko, A. M. Arymurthy, Sleep apnea detection from ecg signal: Analysis on optimal features, principal components, and nonlinearity, in: 2011 5th International Conference on Bioinformatics and Biomedical Engineering, 2011, pp. 1–4. doi:10.1109/icbbe.2011.5780285.

[21] M. Bsoul, H. Minn, L. Tamil, Apnea medassist: Real-time sleep apnea monitor using single-lead ecg, IEEE Transactions on Information Technology in Biomedicine 15 (3) (2011) 416–427. doi:10.1109/TITB.2010.2087386.

[22] H. D. Nguyen, B. A. Wilkins, Q. Cheng, B. A. Benjamin, An online sleep apnea detection method based on recurrence quantification analysis, IEEE Journal of Biomedical and Health Informatics 18 (4) (2014) 1285–1293. doi:10.1109/JBHI.2013.2292928.

[23] K. Karandikar, T. Q. Le, A. Sa-ngasoongsong, W. Wongdhamma, S. T. Bukkapatnam, Detection of sleep apnea events via tracking nonlinear dynamic cardio-respiratory coupling from electrocardiogram signals, in: 2013 6th International IEEE/EMBS Conference on Neural Engineering (NER), 2013, pp. 1358–1361. doi:10.1109/NER.2013.6696194.

[24] A. R. Hassan, M. A. Haque, Computer-aided obstructive sleep apnea screening from single-lead electrocardiogram using statistical and spectral features and bootstrap aggregating, Biocybernetics and Biomedical Engineering 36 (1) (2016) 256–266. doi:https://doi.org/10.1016/j.bbe.2015.11.003.
URL https://www.sciencedirect.com/science/article/pii/S0208521615000868

[25] A. Nishad, R. B. Pachori, U. R. Acharya, Application of TQWT based filter-bank for sleep apnea screening using ECG signals, Journal of Ambient Intelligence and Humanized Computing (May 2018).

[26] S. Martín-González, J. L. Navarro-Mesa, G. Juliá-Serdá, J. F. Kraemer, N. Wessel, A. G. Ravelo-García, Heart rate variability feature selection in the presence of sleep apnea: An expert system for the characterization and detection of the disorder, Computers in Biology and Medicine 91 (2017) 47–58. doi:https://doi.org/10.1016/j.compbiomed.2017.10.004.
URL https://www.sciencedirect.com/science/article/pii/S0010482517303244

[27] C. Varon, A. Caicedo, D. Testelmans, B. Buyse, S. Van Huffel, A novel algorithm for the automatic detection of sleep apnea from single-lead ecg, IEEE Transactions on Biomedical Engineering 62 (9) (2015) 2269–2278. doi:10.1109/TBME.2015.2422378.

[28] P. Kamga, R. Mostafa, S. Zafar, The use of wearable ecg devices in the clinical setting: a review, Current Emergency and Hospital Medicine Reports 10 (3) (2022) 67–72. doi:10.1007/s40138-022-00248-x.
URL https://doi.org/10.1007/s40138-022-00248-x

[29] V. Randazzo, J. Ferretti, E. Pasero, Ecg watch: a real time wireless wearable ecg, in: 2019 IEEE International Symposium on Medical Measurements and Applications (MeMeA), 2019, pp. 1–6. doi:10.1109/MeMeA.2019.8802210.

[30] T. Q. Le, C. Cheng, A. Sangasoongsong, W. Wongdhamma, S. T. S. Bukkapatnam, Wireless wearable multisensory suite and real-time prediction of obstructive sleep apnea episodes, IEEE Journal of Translational Engineering in Health and Medicine 1 (2013) 2700109–2700109. doi:10.1109/JTEHM.2013.2273354.

[31] T. Q. Le, C. Cheng, A. Sangasoongsong, S. T. Bukkapatnam, Prediction of sleep apnea episodes from a wireless wearable multisensor suite, in: 2013 IEEE Point-of-Care Healthcare Technologies (PHT), 2013, pp. 152–155. doi:10.1109/PHT.2013.6461307.

[32] L. Wang, Y. Lin, J. Wang, A RR interval based automated apnea detection approach using residual network, Comput. Methods Programs Biomed. 176 (2019) 93–104.

[33] T. Wang, C. Lu, G. Shen, F. Hong, Sleep apnea detection from a single-lead ECG signal with automatic feature-extraction through a modified LeNet-5 convolutional neural network, PeerJ 7 (e7731) (2019) e7731.

[34] K. Feng, H. Qin, S. Wu, W. Pan, G. Liu, A sleep apnea detection method based on unsupervised feature learning and single-lead electrocardiogram, IEEE Transactions on Instrumentation and Measurement 70 (2021) 1–12. doi:10.1109/TIM.2020.3017246.

[35] T. Chen, H. Yin, Q. H. Nguyen, W. Peng, X. Li, X. Zhou, Sequence-aware factorization machines for temporal predictive analytics, in: 2020 IEEE 36th International Conference on Data Engineering (ICDE), IEEE Computer Society, Los Alamitos, CA, USA, 2020, pp. 1405–1416. doi:10.1109/ICDE48307.2020.00125.
URL `https://doi.ieeecomputersociety.org/10.1109/ICDE48307.2020.00125`

[36] L. Guo, H. Yin, Q. Wang, T. Chen, A. Zhou, N. Quoc Viet Hung, Streaming session-based recommendation, in: Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery Data Mining, KDD '19, Association for Computing Machinery, New York, NY, USA, 2019, p. 1569–1577. doi:10.1145/3292500.3330839.

[37] M. Cheng, W. J. Sori, F. Jiang, A. Khan, S. Liu, Recurrent neural network based classification of ecg signal features for obstruction of sleep

apnea detection, in: 2017 IEEE International Conference on Computational Science and Engineering (CSE) and IEEE International Conference on Embedded and Ubiquitous Computing (EUC), Vol. 2, 2017, pp. 199–202. doi:10.1109/CSE-EUC.2017.220.

[38] P. Henderson, R. Islam, P. Bachman, J. Pineau, D. Precup, D. Meger, Deep reinforcement learning that matters, in: Proceedings of the Thirty-Second AAAI Conference on Artificial Intelligence and Thirtieth Innovative Applications of Artificial Intelligence Conference and Eighth AAAI Symposium on Educational Advances in Artificial Intelligence, AAAI'18/IAAI'18/EAAI'18, AAAI Press, 2018.

[39] W. Lin, K. Hasenstab, G. Moura Cunha, A. Schwartzman, Comparison of handcrafted features and convolutional neural networks for liver MR image adequacy assessment, Scientific Reports 10 (1) (2020) 20336.

[40] C. Guilleminault, S. Connolly, R. Winkle, K. Melvin, A. Tilkian, Cyclical variation of the heart rate in sleep apnoea syndrome. mechanisms, and usefulness of 24 h electrocardiography as a screening technique, Lancet 1 (8369) (1984) 126–131.

[41] W. Yang, J. Fan, X. Wang, Q. Liao, Sleep apnea and hypopnea events detection based on airflow signals using lstm network, in: 2019 41st Annual International Conference of the IEEE Engineering in Medicine and Biology Society (EMBC), 2019, pp. 2576–2579. doi:10.1109/EMBC.2019.8857558.

[42] C. Zhang, D. Hu, T. Yang, Anomaly detection and diagnosis for wind turbines using long short-term memory-based stacked denoising autoencoders and xgboost, Reliability Engineering & System Safety 222 (2022) 108445. doi:https://doi.org/10.1016/j.ress.2022.108445. URL https://www.sciencedirect.com/science/article/pii/S0951832022001107

[43] T. Zhou, W. Wu, L. Peng, M. Zhang, Z. Li, Y. Xiong, Y. Bai, Evaluation of urban bus service reliability on variable time horizons using a hybrid deep learning method, Reliability Engineering & System Safety 217 (2022) 108090. doi:https://doi.org/10.1016/j.ress.2021.108090. URL https://www.sciencedirect.com/science/article/pii/S0951832021005883

[44] N. Rieke, J. Hancox, W. Li, F. MilletarĂ¬, H. R. Roth, S. Albarqouni, S. Bakas, M. N. Galtier, B. A. Landman, K. Maier-Hein, S. Ourselin, M. Sheller, R. M. Summers, A. Trask, D. Xu, M. Baust, M. J. Cardoso, The future of digital health with federated learning, npj Digital Medicine 3 (1) (2020) 119. doi:10.1038/s41746-020-00323-1.
URL https://doi.org/10.1038/s41746-020-00323-1

[45] Q. H. Nguyen, Hybrid Data-Centric and Model-Centric Approach for Robustness Single-Lead ECG Obstructive Sleep Apnea Diagnosis (9 2022).
URL https://github.com/quocnh/Hybrid_Centric

[46] T. Penzel, G. Moody, R. Mark, A. Goldberger, J. Peter, The apnea-ecg database, in: Computers in Cardiology 2000. Vol.27 (Cat. 00CH37163), 2000, pp. 255–258. doi:10.1109/CIC.2000.898505.

[47] V. K. Kapur, D. H. Auckley, S. Chowdhuri, D. C. Kuhlmann, R. Mehra, K. Ramar, C. G. Harrod, Clinical practice guideline for diagnostic testing for adult obstructive sleep apnea: An american academy of sleep medicine clinical practice guideline, Journal of Clinical Sleep Medicine 13 (03) (2017) 479–504. doi:10.5664/jcsm.6506.

[48] D. Benitez, P. Gaydecki, A. Zaidi, A. Fitzpatrick, The use of the hilbert transform in ecg signal analysis, Computers in Biology and Medicine 31 (5) (2001) 399–406. doi:https://doi.org/10.1016/S0010-4825(01)00009-9.
URL https://www.sciencedirect.com/science/article/pii/S0010482501000099

[49] J. Pan, W. J. Tompkins, A real-time qrs detection algorithm, IEEE Transactions on Biomedical Engineering BME-32 (3) (1985) 230–236. doi:10.1109/TBME.1985.325532.

[50] S. S. MEHTA, S. C. SAXENA, H. K. VERMA, Computer-aided interpretation of ecg for diagnostics, International Journal of Systems Science 27 (1) (1996) 43–58. arXiv:https://doi.org/10.1080/00207729608929188, doi:10.1080/00207729608929188.
URL https://doi.org/10.1080/00207729608929188

[51] B.-U. Kohler, C. Hennig, R. Orglmeister, The principles of software qrs detection, IEEE Engineering in Medicine and Biology Magazine 21 (1) (2002) 42–57. doi:10.1109/51.993193.

[52] I. M. Johnstone, B. W. Silverman, Empirical Bayes selection of wavelet thresholds, The Annals of Statistics 33 (4) (2005) 1700 – 1752. doi:10.1214/009053605000000345.
URL https://doi.org/10.1214/009053605000000345

[53] J. Mateo, P. Laguna, Analysis of heart rate variability in the presence of ectopic beats using the heart timing signal, IEEE Transactions on Biomedical Engineering 50 (3) (2003) 334–343. doi:10.1109/TBME.2003.808831.

[54] P. de Chazal, T. Penzel, C. Heneghan, Automated detection of obstructive sleep apnoea at different time scales using the electrocardiogram, Physiol Meas 25 (4) (2004) 967–983.

[55] S. Hochreiter, J. Schmidhuber, Long short-term memory, Neural Comput. 9 (8) (1997) 1735–1780.

[56] Y. LeCun, Y. Bengio, G. Hinton, Deep learning, Nature 521 (7553) (2015) 436–444. doi:10.1038/nature14539.
URL https://doi.org/10.1038/nature14539

[57] M. Ma, Z. Mao, Deep-convolution-based lstm network for remaining useful life prediction, IEEE Transactions on Industrial Informatics 17 (3) (2021) 1658–1667. doi:10.1109/TII.2020.2991796.

[58] J. Bergstra, Y. Bengio, Random search for hyper-parameter optimization, J. Mach. Learn. Res. 13 (null) (2012) 281–305.